

Cerveau d'Audit RAG

Architecture Multi-RAG Orchestree

RGAA Constat Extractor — But 2

Document technique — Comprehensible par un developpeur junior

Zero hallucination

Multi-RAG orchestré

Administrable

OpenRouter

Anti-faux-positifs

[Guide complet](#) · [Architecture](#) · [Procédures](#) · [Administration](#)

Version 1.0 — Mars 2026

Table des Matières

1.	Comprendre le RAG — Explication pour débutant	3
2.	Le Cerveau d'Audit — Vue d'ensemble	4
3.	Les RAG Spécialisés — Qui fait quoi ?	5
4.	La Procédure de Détection du Bon Critère	7
5.	L'Orchestrateur — Comment les RAG collaborent	9
6.	Le Retrieval Hybride — Comment chercher intelligemment	10
7.	Le Context Guard — Zéro hallucination	11
8.	La Formule de Scoring	12
9.	L'Indexation Enrichie du RAG RGAA	13
10.	Administration des Multi-RAG	15
11.	OpenRouter — Configuration LLM	17
12.	Glossaire	18

1. Comprendre le RAG — Explication pour débutant

C'est quoi un RAG ?

RAG signifie **Retrieval-Augmented Generation**. En français : **Génération augmentée par la recherche**. C'est une technique qui permet à un assistant IA de répondre à vos questions en **cherchant d'abord dans vos propres données**, avant de formuler une réponse. Sans RAG, l'IA invente parfois des réponses (on appelle ça une hallucination). Avec RAG, l'IA s'appuie sur des faits réels tirés de vos documents.

Sans RAG Un étudiant qui répond à un examen sans ses notes. Il fait de son mieux, mais il peut se tromper.	Avec RAG Un étudiant qui répond à un examen avec ses notes ouvertes. Il s'appuie sur des faits réels et cite ses sources.
--	---

Les 3 étapes d'un RAG en images

1. QUESTION	2. RECHERCHE	3. REPONSE
L'utilisateur pose une question ou soumet une erreur d'accessibilité	Le système cherche dans les documents pertinents (rapports, RGAA, notes)	L'IA formule une réponse en s'appuyant sur les documents trouvés

Pourquoi plusieurs RAG ?

Dans notre système, il n'y a pas un seul RAG mais plusieurs, chacun spécialisé. C'est comme avoir plusieurs experts dans la même pièce : l'un connaît vos audits passés, l'autre maîtrise le RGAA officiel, le troisième a des notes expertes. Ensemble, ils donnent une réponse bien meilleure qu'un seul expert isolé.

2. Le Cerveau d'Audit — Vue d'ensemble

Le Cerveau d'Audit est le nom donné à l'ensemble du système RAG. Il est composé de **7 couches** qui travaillent ensemble pour produire une réponse fiable, précise, et traçable.

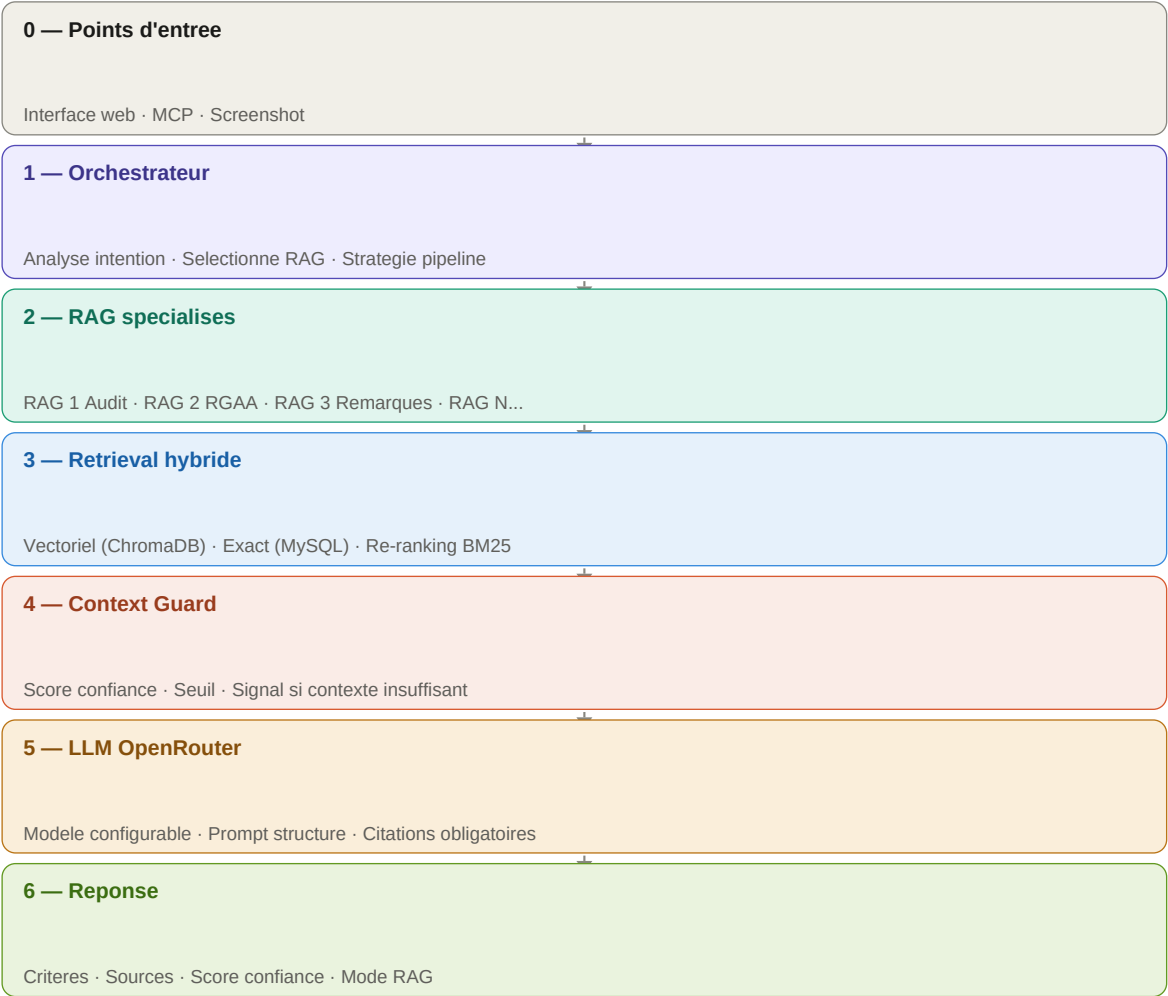


Schéma 1 — Architecture globale en 7 couches du Cerveau d'Audit

Couche 0	Points d'entrée	L'utilisateur peut interagir via l'interface web, le connecteur MCP (dans l'IDE), ou en uploadant une capture d'écran.
Couche 1	Orchestrateur	Le cerveau du système. Il analyse la question, choisit quels RAG interroger et dans quel ordre.
Couche 2	RAG spécialisés	Chaque RAG est expert dans son domaine. Ils répondent en parallèle ou en séquence selon la stratégie choisie.
Couche 3	Retrieval hybride	Combine recherche par sens (vectorielle) et recherche exacte (MySQL) pour trouver les meilleurs documents.
Couche 4	Context Guard	Vérifie que le contexte trouvé est suffisamment fiable avant d'envoyer au LLM. Bloque les hallucinations.
Couche 5	LLM OpenRouter	Le modèle d'IA qui rédige la réponse finale. Configurable : GPT-4o, Claude, Gemini ou autre.

Couche 6	Réponse structurée	La réponse finale inclut toujours : critère(s), sources, score de confiance et mode de réponse.
-------------	-----------------------	---

3. Les RAG Spécialisés — Qui fait quoi ?

Le système dispose de plusieurs RAG, chacun dédié à un type de connaissance précis. Ils sont **administrables** : on peut en ajouter, modifier, désactiver ou supprimer sans toucher au code.

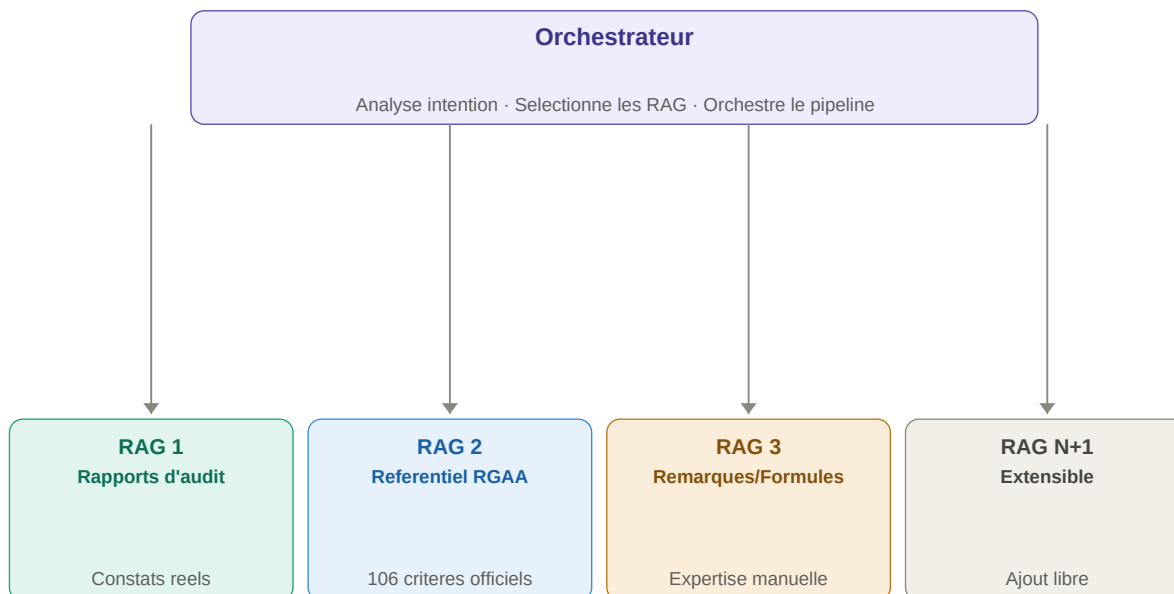


Schéma 2 — Les 4 RAG spécialisés sous l'orchestrateur

RAG 1 Rapports d'audit

C'est la **mine d'or principale**. Il contient tous les constats et solutions issus de vos audits réels passés. Quand une erreur est soumise, c'est lui qu'on interroge en premier, car il contient des patterns déjà validés par des experts humains.

- Collection ChromaDB : rgaa_findings
- Poids scoring : 1.4×
- Seuil similarité : 0.68
- Priorité : 1ère — source primaire

RAG 2 Référentiel RGAA 4.1 officiel

Il contient les **106 critères officiels** du RGAA. Mais pas seulement le texte : chaque critère est indexé avec sa procédure de test structurée, ses conditions de non-applicabilité et son niveau WCAG (A ou AA). C'est le **validateur officiel**.

- Collection ChromaDB : rgaa_referential
- Poids scoring : 1.6×
- Seuil similarité : 0.80 (strict)
- Priorité : 2ème — validation officielle

RAG 3 Remarques & Formules expertes

Il contient des **notes ajoutées manuellement par les experts**. C'est particulièrement utile pour les confusions fréquentes entre critères (ex : 1.1 vs 1.2 vs 1.3). Ces notes améliorent la précision de la détection et enrichissent les réponses.

- Collection ChromaDB : rgaa_notes
- Poids scoring : 1.2×
- Seuil similarité : 0.65
- Priorité : 3ème — enrichissement

**RAG
N+1****Extensible — à créer selon les besoins**

Le système est conçu pour **accepter de nouveaux RAG** sans modification du code. Exemples de RAG futurs : WCAG 2.2 international, règles spécifiques à un client, bibliothèque de composants conformes, documentation technique d'un projet.

- Création via interface admin
- Configuration JSON flexible
- Activation/désactivation à la volée
- Intégration automatique dans le pipeline

4. La Procédure de Détection du Bon Critère

Pourquoi c'est si difficile ?

Une erreur d'accessibilité peut toucher PLUSIEURS critères à la fois. Par exemple, une image sans texte alternatif peut concerner le critère 1.1, 1.2 ou 1.3 selon le contexte. Si le système s'arrête au premier critère trouvé, il peut donner une mauvaise réponse. Il faut TOUJOURS parcourir tous les candidats avant de conclure.

Notre procédure suit exactement le raisonnement d'un auditeur expert humain, enrichi par la force de la recherche dans les données capitalisées :

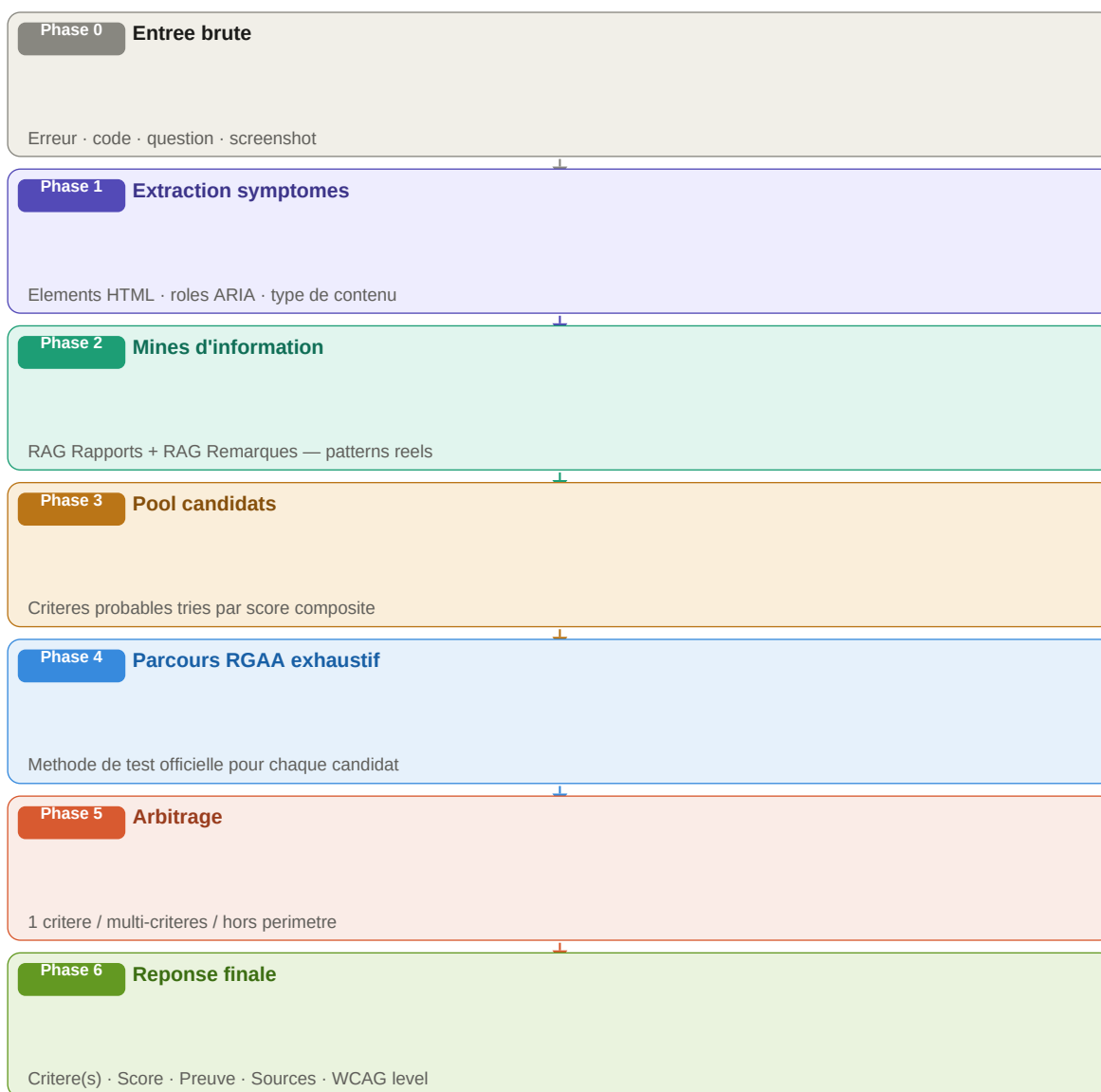


Schéma 3 — Pipeline de détection du bon critère en 7 phases

Détail de chaque phase

Phase 0 — Entrée brute	L'utilisateur soumet une erreur (description textuelle), un extrait de code HTML, une question ou une capture d'écran. À ce stade, on ne sait pas encore quel critère est concerné.
Phase 1 — Extraction des symptômes	Le système extrait automatiquement les informations techniques : quel élément HTML est en cause (img, button, input...), quel rôle ARIA est utilisé, quel type de contenu est impacté (image, formulaire, vidéo...). Ces informations forment le 'profil' du problème.
Phase 2 — Consultation des mines d'information	C'est l'étape la plus importante. On interroge D'ABORD le RAG Rapports (vos audits passés) et le RAG Remarques (notes expertes). Ces sources contiennent des patterns déjà validés par des humains — elles sont bien plus fiables qu'une recherche dans le texte réglementaire pour identifier rapidement les critères candidats.
Phase 3 — Pool de critères candidats	On constitue une liste ordonnée des critères probables (ex : 1.1, 1.2, 1.3, 4.1). Chaque candidat reçoit un score basé sur sa fréquence d'apparition dans les audits passés et sa similarité sémantique avec le problème soumis.
Phase 4 — Parcours RGAA exhaustif	Pour CHAQUE critère candidat, on consulte sa définition officielle ET sa procédure de test dans le RAG RGAA. On vérifie d'abord les conditions de non-applicabilité (pour éviter les faux positifs), puis on applique l'arbre de test structuré. Aucun candidat n'est ignoré avant la conclusion — c'est ce qui évite les erreurs d'attribution.
Phase 5 — Arbitrage	Après le parcours exhaustif, trois cas possibles : (1) Un seul critère confirmé — réponse directe avec haute confiance. (2) Plusieurs critères valides — critère principal + secondaires avec leurs scores respectifs. (3) Aucun critère — le système le signale honnêtement et demande un contexte supplémentaire.
Phase 6 — Réponse finale	La réponse inclut toujours : le(s) critère(s) avec leur référence officielle, le niveau WCAG (A ou AA), la méthode de test applicable, la solution corrective, les sources ayant contribué (quel RAG, quel audit) et le score de confiance global.

5. L'Orchestrateur — Comment les RAG collaborent

L'orchestrateur choisit la **stratégie de collaboration** entre les RAG selon le type de requête. Il existe trois stratégies principales :

A — Séquentielle avec validation	<i>RAG 1 propose → RAG 2 valide → RAG 3 enrichit</i>	Utilisée pour les cas sensibles (analyse code, vérification critère précis). Chaque RAG valide le résultat du précédent avant de passer au suivant. La plus fiable, mais la plus lente.
B — Parallèle avec fusion	<i>RAG 1 + RAG 2 + RAG 3 en simultané → Fusion des résultats</i>	Utilisée pour les questions larges sur l'accessibilité. Tous les RAG répondent en même temps, puis leurs résultats sont fusionnés et classés par score. Plus rapide que la séquentielle.
C — Sélection manuelle	<i>L'utilisateur choisit quels RAG interroger (RAG 1 seul, RAG 1+3, etc.)</i>	Mode expert. L'utilisateur sait exactement quelle source il veut interroger. Utile pour comparer les résultats de différents RAG ou travailler sur un contexte très spécifique.

Exemple concret — Validation croisée pour une image sans alt

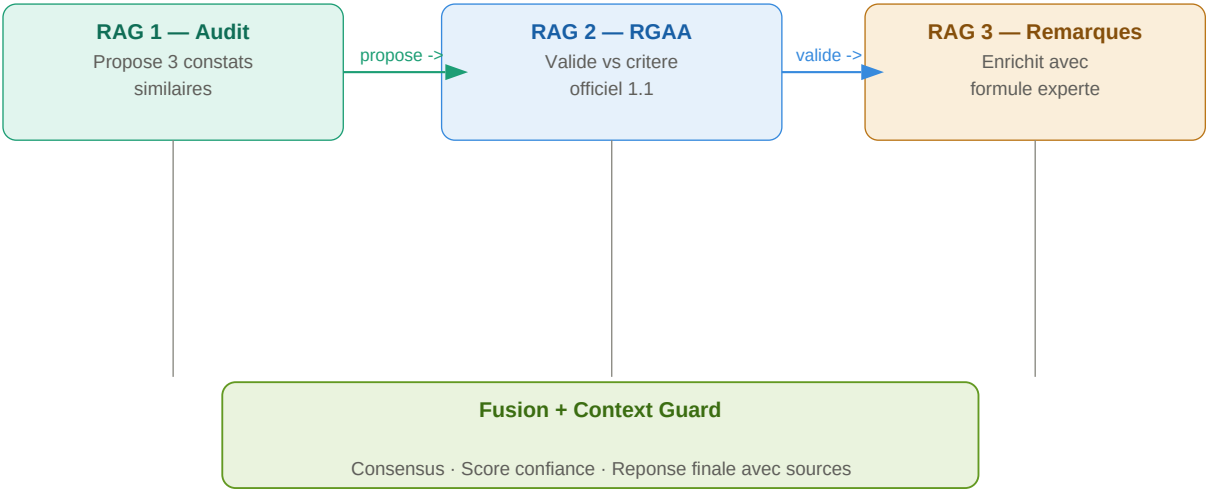


Schéma 4 — Validation croisée entre les 3 RAG pour une image sans texte alternatif

6. Le Retrieval Hybride — Chercher intelligemment

Le retrieval (récupération de documents) est l'étape où le système cherche les informations pertinentes dans les bases de données. Notre système combine **deux types de recherche** complémentaires :

	Recherche vectorielle	Recherche exacte	Re-ranking BM25
Outil	ChromaDB	MySQL	Algorithme sur les résultats
Comment ça marche ?	Transforme le texte en coordonnées	Cherche les correspondances exactes	Reclasse les résultats combinés par pertinence
Exemple	'bouton sans label' trouve aussi 'bouton sans label'	'bouton sans label' trouve exactement 'bouton sans label'	Fusionne et trie les résultats des deux recherches
Idéal pour	Questions libres, descriptions de problèmes	Requêtes RGAA précises, IDs de rapports	Toujours appliqué en dernière étape

Astuce junior Imaginez la recherche vectorielle comme un moteur de recherche Google (trouve par sens) et la recherche exacte comme une base de données de bibliothèque (trouve par référence précise). BM25 est comme un bibliothécaire qui trie les résultats des deux pour vous donner les meilleurs en premier.

7. Le Context Guard — Zéro Hallucination

Le Context Guard est le **mécanisme anti-hallucination** du système. Avant d'envoyer un contexte au LLM pour qu'il rédige la réponse, le Guard vérifie que ce contexte est suffisamment fiable. Si ce n'est pas le cas, il le signale honnêtement plutôt que de laisser l'IA inventer.

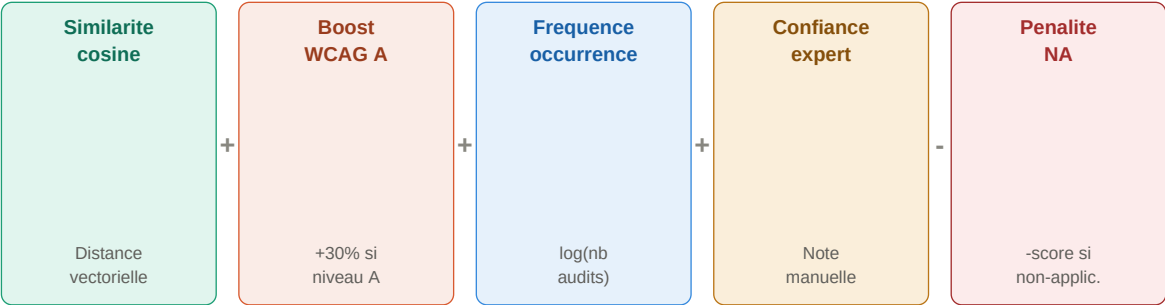
Score minimum requis	Chaque document récupéré a un score de similarité (entre 0 et 1). Si le meilleur score est en dessous du seuil (ex : 0.72), le contexte est jugé insuffisant.
Couverture multi-sources	Le système vérifie qu'au moins 2 sources indépendantes concordent sur la réponse. Une seule source, même avec un bon score, n'est pas suffisante.
Non-applicabilité vérifiée	Avant de retourner un critère, le Guard vérifie si ses conditions d'exclusion sont actives dans le contexte (ex : image décorative → critère 1.1 non applicable).

Les 3 modes de réponse possibles

Mode	Condition	Ce que l'utilisateur voit
RAG complet	Contexte suffisant (score >= seuil, sources concordantes)	Réponse complète avec critères, sources, score de confiance
RAG dégradé	ChromaDB hors ligne ou base vide (cold start)	Réponse LLM seule, signalée clairement comme non sourcée
Contexte insuffisant	Score trop faible ou sources contradictoires	Message transparent : 'données insuffisantes' + ce que le RGAA a

8. La Formule de Scoring

Pour chaque critère candidat, le système calcule un **score de confiance** qui détermine sa position dans la réponse finale. Plus le score est élevé, plus le critère est présenté en premier.



Score final = combinaison de ces 5 composantes → critere le plus pertinent remonte en premier

Schéma 5 — Les 5 composantes du score de confiance

Similarité cosine × poids	La similarité cosine mesure à quel point le document trouvé est proche sémantiquement de la question. Elle varie de 0 (aucun rapport) à 1 (identique). Multipliée par le poids du RAG source (RAG RGAA a un poids plus élevé car plus autoritaire).
Boost WCAG A (+30%)	Si un critère est de niveau WCAG A (obligatoire légalement), son score est multiplié par 1.3. Cela garantit que les non-conformités légalement prioritaires remontent toujours en tête.
Fréquence d'occurrence	Si une erreur a déjà été trouvée 50 fois dans vos audits passés, elle obtient un bonus. $\log(50) = 1.7$, divisé par 5 = +0.34 de bonus. Ça reflète l'expérience réelle.
Confiance expert	Les remarques manuelles ajoutées par les experts peuvent avoir un niveau de confiance (0 à 100). Ce niveau contribue à hauteur de 20% max au score final.
Pénalité non-applicabilité	Si une condition de non-applicabilité est détectée dans le contexte, le critère reçoit une pénalité forte et peut être exclu de la réponse. Évite les faux positifs.

9. L'Indexation Enrichie du RAG RGAA

C'est la différence fondamentale avec l'architecture existante. Chaque critère RGAA n'est plus un simple texte vectorisé — c'est un **objet structuré avec 3 dimensions supplémentaires** :

Dimension 1 — Procédure de test structurée

La procédure de test officielle est encodée comme un arbre conditionnel (JSON), pas comme du texte libre. Le système peut ainsi l'exécuter pas à pas : étape 1 — localiser l'élément, étape 2 — vérifier la condition, etc.

Champ JSON	Valeur exemple	Description
elements_cibles	<code>["img", "area", "input[type=image]"]</code>	Éléments HTML concernés
step_1	<code>"Localiser les images dans la page"</code>	Première étape du test
step_2	<code>"Vérifier la présence de l'attribut alt"</code>	Deuxième étape
pass_if	<code>"alt présent et non vide"</code>	Condition de conformité
fail_if	<code>"alt absent ou vide sans role=presentation"</code>	Condition de non-conformité

Dimension 2 — Conditions de non-applicabilité

Chaque critère peut être non applicable dans certains contextes. Ces conditions sont indexées comme métadonnées et vérifiées EN PRIORITÉ avant tout scoring. Évite les faux positifs.

Champ JSON	Valeur exemple	Description
exclusion_1	<code>"Image décorative (role=presentation)"</code>	Critère 1.1 non applicable
exclusion_2	<code>"Image CSS background only"</code>	Pas d'élément HTML ciblable
exclusion_3	<code>"CAPTCHA (cas spécial)"</code>	Traitement différent
check_first	<code>true</code>	Vérifier avant le scoring
na_message	<code>"Critère non applicable : image décorative"</code>	Message affiché

Dimension 3 — Niveau WCAG et priorité légale

Le niveau WCAG (A ou AA) détermine la priorité légale du critère. Un critère WCAG A non conforme est légalement plus grave. Cette information entre directement dans la formule de scoring.

Champ JSON	Valeur exemple	Description
wcag_level	<code>"A"</code>	Niveau de conformité (A ou AA)
wcag_ref	<code>"1.1.1"</code>	Référence WCAG correspondante
legal_priority	<code>"haute"</code>	Priorité légale calculée
scoring_boost	<code>1.3</code>	Multipliateur appliqué au score

rgaa_version	"4.1.2"	Version de référence
--------------	---------	----------------------

10. Administration des Multi-RAG

Un des objectifs clés est que les RAG soient **entièrement administrables** sans toucher au code. L'interface d'administration permet de créer, modifier, désactiver ou supprimer un RAG selon les besoins du projet.

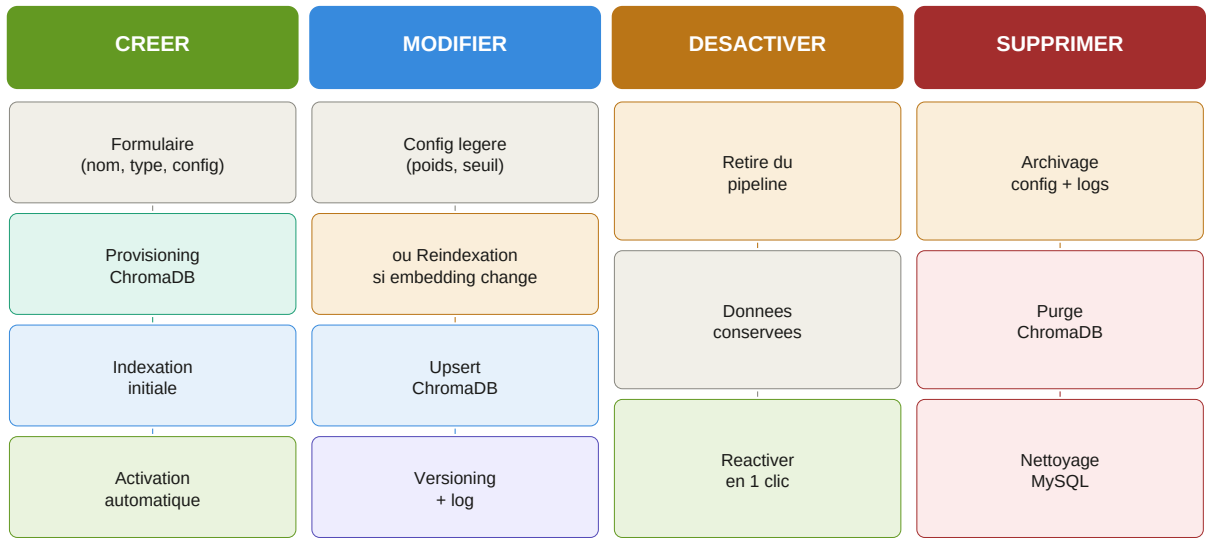


Schéma 6 — Les 4 flux d'administration d'un RAG

Détail des 4 actions

CRÉER un nouveau RAG	
1. Formulaire de création	Nom affiché, slug unique (identifiant technique), type (audit/referential/note/custom), poids dans le scoring, seuil de similarité, nombre de résultats (topK).
2. Provisioning automatique	Le système crée automatiquement la collection ChromaDB correspondante avec les paramètres d'embedding choisis.
3. Alimentation des données	Trois modes : upload de fichiers, scraping d'une URL (ex : site RGAA officiel), ou ajout manuel note par note.
4. Indexation initiale	Les documents sont vectorisés et indexés dans ChromaDB. Le RAG passe en statut 'indexing' puis 'active'.

MODIFIER un RAG existant	
Config légère (sans réindexation)	Modifier le poids, le seuil de similarité ou le topK. Effet immédiat sur les prochaines requêtes.
Config lourde (avec réindexation)	Changer le modèle d'embedding force une réindexation complète. Le RAG passe temporairement en 'indexing'.
Ajout/suppression de contenu	Upsert dans ChromaDB via la signatureHash. Les documents existants sont mis à jour, pas dupliqués.
Versioning	Chaque modification est loggée avec horodatage. Rollback possible vers la version précédente.

DÉSACTIVER un RAG	
-------------------	--

Impact immédiat	Le RAG est retiré du pipeline de l'orchestrateur. Les requêtes en cours se terminent normalement.
Données préservées	La collection ChromaDB et les données MySQL sont conservées intactes.
Réactivation facile	Un clic suffit pour réactiver le RAG. Si les données sont intactes, disponible immédiatement.
Règle de garde	Impossible de désactiver simultanément tous les RAG. Au moins 1 RAG actif est toujours requis.

SUPPRIMER un RAG

Confirmation obligatoire	Le système affiche la liste de tous les pipelines et requêtes qui utilisent ce RAG.
Archivage préalable	La configuration, les logs et les statistiques sont exportés avant la suppression.
Purge complète	La collection ChromaDB est supprimée. L'entrée rag_sources est retirée de MySQL.
Règle de garde critique	RAG 1 (Rapports) et RAG 2 (RGAA) ne peuvent pas être supprimés s'ils sont les seuls de leur type.

11. OpenRouter — Configuration du LLM

OpenRouter est un service qui donne accès à **tous les grands modèles d'IA** (GPT-4o, Claude, Gemini, Mistral...) via une seule API. Le modèle utilisé est **configurable sans redéploiement** depuis l'interface d'administration.

Paramètre	Valeur par défaut	Description
Modèle principal	gpt-4o (OpenAI)	Modèle utilisé pour générer les réponses
Modèle fallback	claude-sonnet (Anthropic)	Utilisé si le modèle principal est indisponible
Température	0.1	Très basse : réponses précises et reproductibles (pas créatives)
Max tokens	1000	Limite la longueur des réponses
Timeout	30 secondes	Délai maximum avant d'utiliser le modèle fallback
Citations obligatoires	true	Le LLM DOIT citer les sources dans sa réponse
Langue	fr	Langue de réponse forcée

Pourquoi la température à 0.1 ?

Dans le domaine de l'accessibilité, on veut des réponses précises et basées sur des faits réels, pas des réponses créatives ou variables. Une température proche de 0 force le modèle à s'en tenir strictement au contexte fourni. Une température élevée (proche de 1) produirait des réponses plus variées mais moins fiables — inacceptable pour un outil professionnel.

Instructions anti-hallucination dans le prompt

Le LLM reçoit des instructions strictes dans son prompt système : (1) Citer obligatoirement la source de chaque affirmation. (2) Mentionner explicitement si une information provient de la base de données ou de sa connaissance générale. (3) Dire clairement 'je ne sais pas' si le contexte est insuffisant. (4) Ne jamais extrapoler au-delà du contexte fourni par le RAG.

12. Glossaire — Les termes clés expliqués

Terme	Signification	Explication
RAG	Retrieval-Augmented Generation	Technique permettant à une IA de chercher dans des documents réels avant de répondre.
Embedding	Vectorisation de texte	Transformation d'un texte en une liste de nombres qui représente son sens. Deux textes proches ont des vecteurs proches.
ChromaDB	Base de données vectorielle	Base de données spécialisée pour stocker et rechercher des embeddings. Permet la recherche par sens.
Similarité cosin	Mesure de proximité	Score entre 0 et 1 indiquant à quel point deux vecteurs pointent dans la même direction (= ont un sens similaire).
BM25	Algorithme de re-ranking	Algorithme statistique qui reclasse les résultats de recherche en tenant compte de la fréquence des mots.
Hallucination	Erreur de l'IA	Quand une IA invente une information fausse présentée comme vraie. Le Context Guard existe pour éviter ça.
Upsert	Opération base de données	Insert + Update : crée l'entrée si elle n'existe pas, la met à jour si elle existe déjà.
topK	Paramètre de retrieval	Nombre maximum de documents retournés par la recherche. Ex : topK=8 retourne les 8 documents les plus pertinents.
WCAG	Web Content Accessibility Guidelines	Standard international d'accessibilité web. Le RGAA s'appuie sur WCAG. Niveau A = obligatoire, AA = renforcé.
Non-applicabilité	Condition d'exclusion	Cas où un critère RGAA ne s'applique pas (ex : image décorative pour critère 1.1). Doit être vérifié avant le scoring.
Orchestrateur	Chef d'orchestre des RAG	Composant qui décide quels RAG interroger, dans quel ordre et avec quelle stratégie.
Pipeline	Chaine de traitement	Séquence d'étapes de traitement d'une requête, de l'entrée jusqu'à la réponse finale.
MCP	Model Context Protocol	Protocole standard permettant à des outils IA externes (Claude, Cursor) d'utiliser les fonctions du système.
Fire-and-forget	Tâche asynchrone	Lancer une tâche en arrière-plan sans attendre sa fin. Ex : indexation ChromaDB après un import réussi.
Slug	Identifiant URL	Version simplifiée d'un nom, sans espaces ni accents. Ex : 'rag-rapports-audit'. Utilisé comme clé unique.

Ce document est le fondement du But 2 — Evolution du RAG vers un niveau professionnel. Il couvre l'architecture complète, les procédures de détection multi-critères alignées sur les pratiques réelles des auditeurs, l'administration des multi-RAG, et le système anti-hallucination. La prochaine étape est le **Cahier des Charges Fonctionnel (But 1)** qui s'appuiera sur cette architecture pour définir les exigences fonctionnelles complètes du projet.